P03b Wall Jump

CRE132 worksheet

What you'll make

Two more moves for P03's `AdvancedController`: a **wall slide** (push into a wall while falling and you slide down it slowly) and a **wall jump** (jump while sliding to kick off the wall). With them you can climb the tall shaft at the start of the level by bouncing between its walls.

By the end you'll know how a raycast checks for a wall in one direction, and why a short timer that ignores the controls makes the wall jump work.

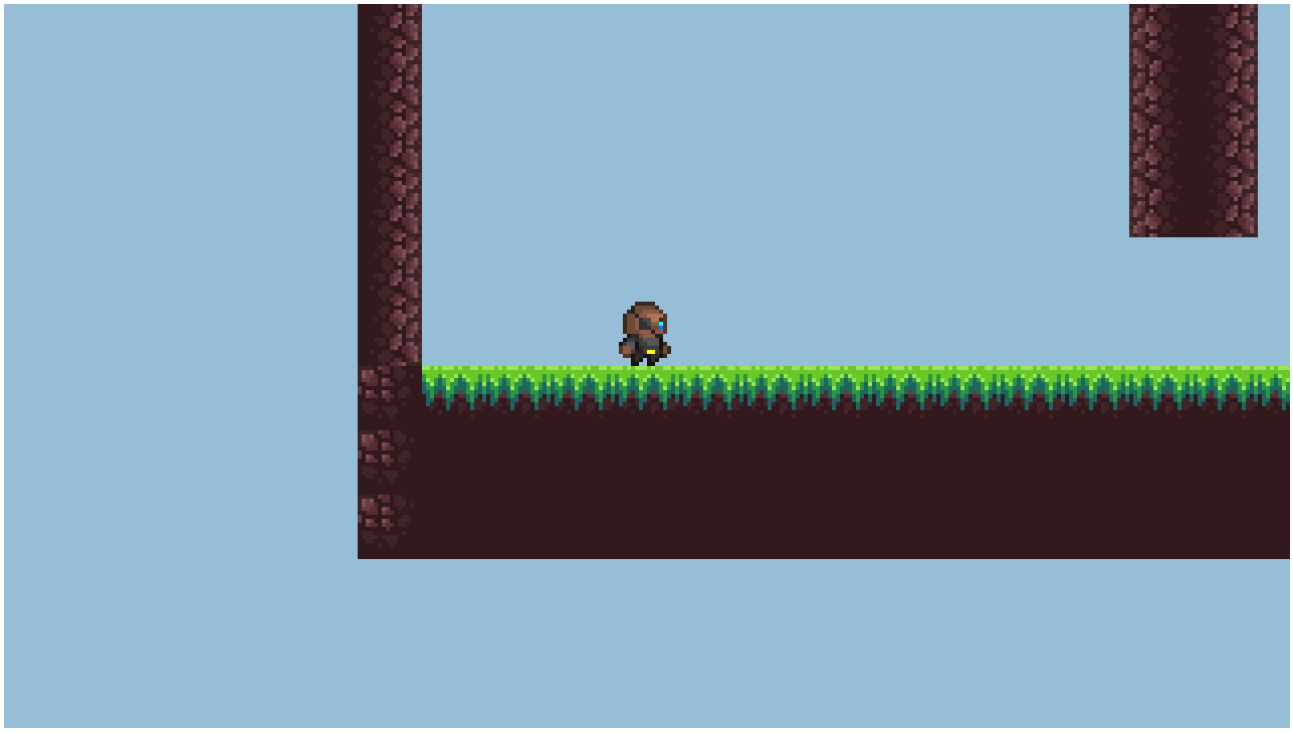


Figure 1: The start of the level: the wall-jump shaft on the left.

Before you start

1. Unzip `CRE132_P03b_WallJump_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P03b_WallJump_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P03b_WallJump`.
3. Press **Play**. Everything from P03 works: run, jump, double jump and dash (K). Jump into the shaft and hold towards a wall: you just fall. Press Play again to stop.

The **Player** already has P03's **Advanced Controller** and **Dash**. You'll add the wall moves to `AdvancedController`, in `Assets/_game/Scripts/P03_AdvancedMovement`. Double-click it to open it.

1. Wall check and wall slide

To slide, the script needs to know whether a wall is there, on the side the player faces. Add these fields below `extraJumps`:

```
    [Header("Wall Slide and Wall Jump")]
    [SerializeField] private float wallCheckDistance = 0.5f;
    [SerializeField] private float wallSlideSpeed = 2f;
```

Add a bool just above the `facingDirection` comment (below `private int extraJumpsLeft;`):

```
    private bool isWallSliding;
```

In `FixedUpdate`, below `velocity.x = horizontalInput * moveSpeed;` and above `bool wantsToJump`, add:

```
        // 2. Wall slide: touching a wall in the air, pushing into it, and falling.
        Vector2 wallCheckStart = new Vector2(transform.position.x,
            transform.position.y + 0.5f);
        RaycastHit2D wall = Physics2D.Raycast(wallCheckStart,
            new Vector2(facingDirection, 0f), wallCheckDistance, groundLayers);
        bool pushingIntoWall = horizontalInput * facingDirection > 0.1f;
        if (wall.collider != null && !isGrounded && pushingIntoWall && velocity.y < 0f)
        {
            isWallSliding = true;
            if (velocity.y < -wallSlideSpeed)
            {
                velocity.y = -wallSlideSpeed;
            }
        }
        else
        {
            isWallSliding = false;
        }
```

Last, at the end of `OnDrawGizmosSelected` (below `Gizmos.DrawWireSphere`), add a gizmo for the wall check:

```
        // The wall check ray.
        Gizmos.color = Color.cyan;
        Vector3 wallCheckStart = transform.position + new Vector3(0f, 0.5f, 0f);
        Gizmos.DrawLine(wallCheckStart,
            wallCheckStart + new Vector3(facingDirection * wallCheckDistance, 0f, 0f));
```

- **`Physics2D.Raycast`** fires a ray from the middle of the player (0.5 up from the feet) towards the facing side, `wallCheckDistance` long, and only hits things on the Ground layer. `wall.collider` is `null` when it hits nothing.
- **`facingDirection`** is the one you added in P03 for the dash: 1 for right, −1 for left.
- **Pushing into the wall:** `horizontalInput * facingDirection` is positive only when the input and the facing point the same way.
- **Four things must all be true** to slide: a wall, not on the ground, pushing into it, and falling. Then the fall speed is capped at `wallSlideSpeed`.

Add it: save and go back to Unity. The new fields appear on the Player's **Advanced Controller** with **Wall Check Distance 0.5** and **Wall Slide Speed 2**.

Test it: jump into the shaft and hold towards a wall: you slide down slowly. Select the **Player** while playing: the cyan line in the Scene view shows the wall check and flips when you turn.

2. Wall jump

While sliding, Jump should kick you up and away from the wall. Add these fields below `wallSlideSpeed`:

```
    [SerializeField] private Vector2 wallJumpVelocity = new Vector2(7f, 13f);
    // After a wall jump, ignore left/right input briefly, or pushing towards the wall
    // would cancel the kick.
    [SerializeField] private float wallJumpControlLockTime = 0.2f;
```

Add a counter below `private float facingDirection = 1f;`:

```
    private float controlLockCounter;
```

In `FixedUpdate`, replace the line `velocity.x = horizontalInput * moveSpeed;` with:

```
        // Left/right, unless a wall jump has just locked the controls.
        if (controlLockCounter > 0f)
        {
            controlLockCounter = controlLockCounter - Time.fixedDeltaTime;
        }
        else
        {
            velocity.x = horizontalInput * moveSpeed;
        }
```

Then add the wall jump between the normal jump and the double jump: below the normal jump's closing brace and above `else if (wantsToJump && extraJumpsLeft > 0)`:

```
        else if (wantsToJump && isWallSliding)
        {
            // 3. Wall jump: up and away from the wall, then face the other way.
            velocity.x = -facingDirection * wallJumpVelocity.x;
            velocity.y = wallJumpVelocity.y;
            facingDirection = -facingDirection;
            controlLockCounter = wallJumpControlLockTime;
            isWallSliding = false;
            jumpBufferCounter = -1f;
        }
```

- **`-facingDirection`** is away from the wall. The kick is 7 sideways and 13 up, and the player then faces the other way.
- **The control lock:** for 0.2 seconds left/right input is ignored. You're still holding towards the wall, and without the lock that would cancel the kick at once.
- **Order matters:** ground jump first, then wall jump, then double jump. Because it's an `else if` chain, only one of them happens, and a wall jump doesn't use up your double jump.

Add it: the new fields show **Wall Jump Velocity (7, 13)** and **Wall Jump Control Lock Time 0.2**.

Test it: bounce between the two walls of the shaft to climb it and get over the tall wall. Then set **Wall Jump Control Lock Time** to **0** and try again: the kick barely moves you. Set it back to **0.2**.

Check your work

- ☐ Holding towards a wall while falling slides you down slowly.
- ☐ Jump while sliding kicks you up and away from the wall; you can climb the shaft.
- ☐ After a wall jump you still have your double jump.

- ☐ The cyan wall check line shows on the selected Player and flips when you turn.
- ☐ Double jump and dash still work as in P03.
- ☐ Compare with `CRE132_P03b_WallJump_Finished` if something behaves differently.

Key points

- **A raycast answers "is something there?"** in one direction, at a set distance, on the layers you choose.
- **Small timers like the control lock** make moves feel deliberate instead of glitchy.
- **The order of the `if` / `else if` chain** decides which jump wins: ground, then wall, then double jump.

If something goes wrong

- **"The name 'isWallSliding' does not exist" (or `controlLockCounter`, `wallCheckDistance`...):** a field is missing, or its name is spelt differently from where it's used.
- **"'else' without 'if'" or similar errors:** the wall jump `else if` has been pasted somewhere other than straight after the closing brace of the normal jump.
- **The wall slide never happens:** you must be in the air, falling and holding towards the wall, and the wall must be on the **Ground** layer. Check the cyan gizmo reaches the wall.
- **The player wall jumps but heads straight back to the wall: Wall Jump Control Lock Time** is 0, or the control lock code didn't replace the `velocity.x` line.

Try it

1. Make wall jumps refill the double jump.
2. Let the player wall jump only a few times before touching the ground. (Hint: a counter like `extraJumpsLeft`, refilled on the ground.)